

第 6 週目事前課題 作業ログ

25G1144 脇阪隼人

2026 年 6 月 24 日

Arduino オーケストラ

1 進捗サマリー (要約)

- 全体進捗率: Ptype2 で SA / SP の和音対応・演奏順不具合修正・中級アレンジ譜への更新が完了し、Ptype3 として 16 分音符単位の Tick 送信方式への移行（整合性修正）も完了。実機書き込み・通し演奏での最終検証が残っている。
- マイルストーン達成状況: 達成 (Slave Arduino・Slave Processing の主要機能追加分、Ptype3 での Tick 解像度変更分)
- 主要成果:
 - SA_ptype1.ino のコンパイルエラー (配列リファクタの不整合、UNO R4 WiFi での volatile String 起因エラー) を特定・修正
 - SA / SP で主旋律と対旋律を同時演奏できる機能を追加 (4 値シリアル送信 + Minim でのミックス再生)
 - SA に myPlayOrder==4 (ドラム) の発音ロジックを追加。ハイハット/スネアを偶数 Tick ごとに交互発音
 - DrumProcessing_new.pde を新しい SA 仕様に合わせて修正し、未インストールライブラリ (processing.sound) 依存と API 誤用 (Noise.setAmplitude) を解消
 - ファイル構成を MA/MP1・MP2/SA/SP_Piano・SP_Drum に再編し、GitHub へ push
 - Ptype2 の SA 楽譜配列を「主旋律 1 音」から「主旋律・対旋律それぞれ最大 3 音の和音」対応の二次元配列に拡張し、送信フォーマットを 4 値 (音階 0,1,2, 音の長さ) に変更
 - MP1/MP2/else.MP_ptype2 の instNames 順序不一致 (木琴とフルートの演奏順が入れ替わって送信される不具合) を特定・修正
 - もりのくまさん中級アレンジ譜 (gaku_hu_tyukyu.pdf) の全 13 小節を採譜し、SA の楽譜配列を pitch_to/pitch_he (104 要素) に全面更新
 - Ptype3 を作成し、楽譜解像度を 8 分音符基準から 16 分音符基準 (208 要素) へ拡張した上で、MA/SA/SP_Drum の Tick 送信間隔・演奏順の開始・合流タイミング・ドラム発音間

隔・総発音数を整合させる修正を実施

・重大課題（影響度：高）：

- ローカル環境に arduino-cli 等のコンパイラがなく、Claude Code での修正はコードレビューのみで実機・実コンパイル未検証
- MA/MP 側（Master Arduino / Processing）はメンバーが独自に大幅改修中で、SA/SP とのプロトコル整合性は未確認
- ピアノ実機ボードの INST_ID 設定が誤っている可能性（演奏順 1 番手で送信しているにもかかわらず輪奏部分を演奏する現象）を特定したが、実機側の `#define INST_ID` 値の確認・修正はユーザー側作業として未対応

2 作業ログ（詳細トラッキング）

2.1 SA_ptype1.ino のコンパイルエラー修正

作業日時：2026 年 6 月 18 日 作業時間：4 時間状態：完了 進捗率：100%

依存関係・ブロッカー：なし

作業内容：

楽譜配列を 2 次元（和音用）から 1 次元へリファクタリングした際、`myPlayOrder==2/3` の分岐が削除済みの `pitch_sub/duration_sub/SCORE_LENGTH_SUB` を参照したままになっており、かつ `pitch_main[index][0]` のような 2 次元添字アクセスが残っていたためビルド不能になっていた。単一の `pitch[]/duration[]` 配列と `startTick` オフセットによるロジックに統一して解消した。

その後、UNO R4 WiFi（Renesas コア）に書き込んだ際に `passing 'volatile arduino::String' as 'this' argument discards qualifiers` が発生。Renesas コアの String クラスには `volatile` 修飾のメンバ関数が無いことが原因であったため、受信バッファを `volatile char` 配列 + 長さ変数に置き換え、`loop()` 側で通常の String へ変換する方式に変更して解消した。

2.2 演奏順（myPlayOrder）ごとの開始位置・合流ロジックの調整

作業日時：2026 年 6 月 18 日 作業時間：2 時間状態：完了 進捗率：100%

依存関係・ブロッカー：なし

作業内容：

`myPlayOrder==2,3` が演奏を開始する Tick と、楽譜上のどの要素目から始めるかを `startTick / indexOffset` という 2 変数で一般化。要望に応じて開始要素を 37 → 33 要素目に調整した後、`TickCount=66`（`myPlayOrder==1` が 65 要素目を演奏するタイミング）以降は `myPlayOrder==1` と同じ index 進行に合流するよう変更した。

2.3 主旋律・対旋律の同時演奏機能の追加 (SA / SP)

作業日時：2026 年 6 月 18 日 作業時間：2 時間状態：完了 進捗率：100%

依存関係・ブロッカー：対旋律データは過去に revert されたコミット（対旋律の楽譜を作成）の pitch_sub/duration_sub を再利用することで合意

作業内容：

SA_ptype1.ino で主旋律 (pitch_main/duration_main) と対旋律 (pitch_sub/duration_sub) を毎 Tick で計算し、mainPitch,mainDuration,subPitch,subDuration の 4 値を Serial で送信するよう変更。SP_ptype1.pde (現 SP_Piano) 側は playVoice() を追加し、2 音をそれぞれ out.playNote() に渡すことで、Minim の出力バス上でミックスして同時再生できるようにした。

2.4 ドラム機能 (myPlayOrder==4 / INST_ID==4) の追加

作業日時：2026 年 6 月 18 日 作業時間：1 時間状態：完了 進捗率：100%

依存関係・ブロッカー：既存の DrumsSlave_ver2.ino を参考実装として利用

作業内容：

SA_ptype1.ino に handleDrum() を追加し、INST_ID==4 のときだけ通常楽器の処理から分岐させ、他楽器 (INST_ID!=4) の既存ロジックには影響を与えないようにした。要件：(1) ハイハット (drumType=2,vel=80) とスネア (drumType=1,vel=110) の 2 種類、(2)i2cReceiveCount-1 を TickCount (0 始まり) として扱う、(3)TickCount が偶数のときだけ発音、(4) 発音ごとにハイハット→スネアと交互、(5)TickCount==97 (i2cReceiveCount==98) で isPlaying=false にして終了、(6) 出力フォーマットは DrumsSlave_ver2.ino と同じ drumType,velocity,BPM、をすべて満たす実装とした。

2.5 DrumProcessing_new.pde の SA 仕様対応・エラー修正

作業日時：2026-06-18 状態：完了 進捗率：100%

依存関係・ブロッカー：なし

作業内容：

旧 DrumProcessing_new.pde は DrumsSlave_ver2.ino 向けに書かれており、START/END/"DrumSlave ready" という明示的なステータス文字列を前提にしていたが、新しい SA の handleDrum() はこれらを送信せず、デバッグ用の行 ("I2C: ..."等) も同じシリアルに混在する。カンマ区切りでちょうど 3 項目の行のみを演奏データとして扱うように修正し、isPlaying/hitCount の管理もステータス文字列に依存しない自己完結型のロジックに変更した (偶数 Tick のみ発音のため、全 98Tick 中の総発音数は 49 回であることも反映)。

その後、実行時に 2 つのエラーが発生：

- No library found for processing.sound : Processing Sound ライブラリが未インストール。
本プロジェクトの他の SP ファイルで既に動作確認済みの ddf.minim に置き換え、新規ライブ

ラリのインストールを不要にして解消。

- The function `setAmplitude(float)` does not exist: Minim の Noise クラスには `setAmplitude()` メソッドが存在せず、振幅はコンストラクタで指定する仕様であったため、`new Noise(amplitude, Noise.Tint.WHITE)` の形に修正して解消。

2.6 SA / SP の楽譜構造を和音対応の二次元配列化し、送信フォーマットを 4 値に拡張

作業日時：2026 年 6 月 23 日 作業時間：4 時間状態：完了 進捗率：100%

依存関係・ブロッカー：対旋律 (`pitch_sub/duration_sub`) の実データは、過去に revert されたコミット（対旋律の楽譜を作成）のものを復元して使用

作業内容：

AI に楽譜画像を pdf 形式で渡し、画像解析によって得た楽譜の midi 番号や持続時間を配列として作成してもらった。実際には楽譜仕様通りの演奏を行うことができなかったため、独自に新しく 2 次元楽譜配列を作成した。SA の楽譜配列を「主旋律 1 音のみ」の 1 次元配列から、主旋律 `pitch_main`・対旋律 `pitch_sub` それぞれが 1 要素につき最大 3 音を持つ二次元配列に拡張。音の長さ配列 (`duration_main/duration_sub`) は 1 次元のまま維持した。`myPlayOrder==1,2,3` のときのシリアル送信フォーマットを、主旋律・対旋律をそれぞれ別行で「音階 0, 音階 1, 音階 2, 音の長さ」の 4 値に変更 (`myPlayOrder==4` は変更なし)。SP_Piano/SP_Mokkin/SP_flute 側の `serialEvent()` を 4 値パースに対応させ、受信した最大 3 音 (0 は休符) を同じ `duration` で同時発音するように修正した。

2.7 Tick 送信を 8 分音符基準から 16 分音符基準への変更

作業日時：2026 年 6 月 24 日 作業時間：3 時間状態：完了（実機検証は未実施） 進捗率：100% (コード変更分)

依存関係・ブロッカー：なし

作業内容：

楽譜の解像度が 8 分音符単位から 16 分音符単位 (104 → 208 要素) に変更されたことに合わせて、演奏システム全体の整合性を取るための修正を実施した。

- `MA_ptype1.ino` : Tick 送信間隔の計算式 `30000UL/BPM` (8 分音符の ms) を `15000UL/BPM` (16 分音符の ms) に変更 (初期設定・BPM 同期・通常送信の計 3 箇所+最終待機の 1 箇所)。Tick 終了判定を `tickCount>=104` から `tickCount>=208` に変更。
- `SA_ptype2.ino` : `myPlayOrder==2,3` の演奏開始位置を新しいタイミング (`tickCount=83` で楽譜 75 要素目から開始、`tickCount=139` で 138 要素目に合流) に合わせて `startTick=37` → `82`, `indexOffset=32` → `74`、合流条件を `i2cReceiveCount>=66` → `>=139` に変更。ドラム専用処理 (`handleDrum()`) は Tick が 2 倍速くなった分、従来と同じ実時間でハイハット／スネア交互発音間隔 (4 分音符ごと) を保つため判定を `%2==0` → `%4==0` に変更し、終了判定を `tickCount==103` → `207` に変更。

変更後、myPlayOrder==1/2,3・ドラムの境界 Tick 値（83, 138, 139, 207, 208 等）について index 計算を手計算でトレースし、オーバーラン・ズレが無いことを机上で確認した。実機への書き込み・Processing 起動・通しでの演奏確認は未実施。

3 課題管理

表 1 課題一覧

課題	発生原因	影響度	優先度	対策
楽器ごとの音量比設定	同一 PC、複数 Processing での結合テスト	中	中	1 つの楽器につき 1 つの PC を

※影響度＝プロジェクトへのインパクト ※優先度＝対応の緊急性

4 AI 利用状況と検証（再現性重視）

4.1 利用内容

- ・利用目的：デバッグ（コンパイルエラー・演奏順不具合の原因特定）／実装（主旋律・対旋律の和音対応、楽譜の全面更新、ドラム機能追加、Tick 解像度変更）／レビュー（Processing 側ライブラリ・API 不整合の修正、MP 間の楽器順序整合性確認）
- ・入力（プロンプト要旨）：
 - 修正前後の SA_ptype1.ino の差分を提示し、問題点と原因の特定を依頼
 - UNO R4 WiFi 書き込み時のコンパイルエラーメッセージをそのまま提示し修正を依頼
 - 「myPlayOrder==2,3 を 1 番手の進行に 65 要素目で合流させたい」等、楽譜進行の仕様変更を自然言語で指示
 - 「主旋律と対旋律を同時に演奏するように SA・SP を変更せよ」「Drumslave_ver2.ino を参考に myPlayOrder==4 のドラム機能を追加せよ」という要件付きの実装依頼
 - Processing 実行時のエラーメッセージ（ライブラリ未検出、メソッド不存在）をそのまま提示し修正を依頼
 - 楽譜構造および送信データ形式変更に伴うシステム全体の整合性修正
- ・出力概要：
 - コンパイルエラーの原因（配列リファクタの取り残し、volatile String 非対応）の説明と修正コード
 - SA/SP のシリアル通信フォーマットを 4 値・3 値に拡張する実装
 - INST_ID==4 専用のドラム発音ロジック（handleDrum 関数）
 - processing.sound 依存を ddf.minim に置き換えた DrumProcessing_new.pde
 - Tick 送信間隔・myPlayOrder 開始・合流タイミング・ドラム発音間隔の修正コード

4.2 検証方法

※第三者が再現できるレベルで記述

- 検証手法：実機での回路作成・結合テスト
- 検証手順：
 1. 実機・実行環境でテストを行い、実際の動作やエラーコードを確認

4.3 ファクトチェック結果

- 一致点：
 - 自分で設定した演奏順ごとの楽譜再生位置を基に AI が作成したコードでは、もりのくまさんの楽譜仕様通りの輪奏を行うことができた。
- 不一致・疑義：
 - AI が作成した楽譜では、もりのくまさんとして演奏することができなかった。
- 最終判断：大枠は AI が作成したコードを採用し、適宜個人で修正を行なった。楽譜に関しては AI の出力は採用せず、全て個人で作成した。
- 根拠：
 - 機能や関数ごとに AI の出力を実機でのテストを行い、意図通りの動作をすること確認できた。AI の出力した楽譜では、音階や再生する音の数が異なり、意図通りの演奏ができなかったこと。

5 次回計画

- 目標：
 - 実機（Arduino UNO R4 WiFi ×複数台、PC 上の Processing）で MA/MP/SA/SP を結合し、主旋律・対旋律・ドラムを含む全体演奏を通して確認する
 - 楽器 Arduino 一つにつき一台の PC（Processing）を使用したテストを行うことで、楽器ごとの適切な音量比を設定する。
- 達成基準：
 - もりのくまさんの楽曲が冒頭から終端まで、ピアノ・木琴・フルート（主旋律+対旋律）とドラムが破綻なく同時再生される
 - 各楽器の適切な音量比を確定させる。
- 期限：次回授業

6 その他

なし

7 付録

GitHub リポジトリ <https://github.com/Riarumogura/Hackathon1/tree/main/group7>